UNIVERSIDAD ALFONSO X EL SABIO

BUSINESS & TECH

GRADO EN INFORMÁTICA

147191



TRABAJO FIN DE GRADO

Pipeline de procesamiento digital de señales

Nicolás Jiménez Rodríguez

JUNIO 2025



UNIVERSIDAD ALFONSO X EL SABIO

BUSINESS & TECH

GRADO EN INFORMÁTICA - 147191

Pipeline de procesamiento digital de señales

Alumno: Nicolás Jiménez Rodríguez

N.P.: 147191

Tutor académico: Ana María González Plaza

Fecha de presentación: 01/07/2026

Breve descripción: Desarrollo e implementación de una pipeline de procesamientos y transmisión de archivos I/Q mediante SDR.



ÍNDICE

1- Introducción

1.1- Contexto y Motivación

1.2- Estado del arte

1.3- Problema a solucionar

1.4- Objetivo

1.5- Alcance y limitaciones

2- Base teórica

2.1- Radiofrecuencia (RF)

2.1.1- Conceptos

2.1.2- Sistemas de comunicación

2.2- Representación I/Q

2.2.1- Componente I y Q

2.2.2- Amplitud y Fase

2.2.3- Plano I/Q

2.3- Modulación

2.3.1- Modulación digital

2.3.2- Modulación de Amplitud en Cuadratura (QAM)

2.4- Procesamiento Digital de Señales (DSP)

3- Tecnología y herramientas utilizadas

3.1- Entorno software

3.1.1- Python

3.1.2- Matlab

3.1.3- Entorno de transmisión

3.2- Hardware

3.2.1- SDR

3.2.2- Analizador de espectro

4- Diseño de la pipeline

5- Implementación del procesamiento

5.1- Archivos IQ

5.2 - Visualización

5.3- Procesamiento de archivos

5.4- Validación (visualización)

6- Implementación de la transmisión

6.1- Configuración de entorno

6.2- Desarrollo de código de transmisión

6.3- Transmisión RF

6.4- Validación

7- Mejoras futuras

8- Conclusión

9- Bibliografía

10- Anexo

1- Introducción

1.1 Contexto y motivación

En los últimos años, las tecnologías de Software Defined Radio (SDR), han ido ganando mucha popularidad e importancia de manera global en el ámbito de las telecomunicaciones y procesamiento digital, además de otros sectores como en Defensa [1] . Esto se debe a su flexibilidad y a su gran capacidad para implementar diferentes tipos de sistemas de comunicación mediante software, lo que hace que cada vez sean más usados tanto en investigaciones como en soluciones industriales.

A diferencia de las radios convencionales, en las que el procesamiento de la señal se realiza mediante circuitos hardware diseñados para una función concreta, una SDR convierte la señal de radiofrecuencia en muestras digitales IQ al inicio de la cadena, dejando que el resto del procesamiento (filtrado, demodulación o decodificación) sea realizado por software. Esto permite que un mismo dispositivo pueda adaptarse a distintos protocolos y aplicaciones simplemente modificando el software utilizado, sin necesidad de cambiar el hardware.

Además, las SDR ofrecen otras ventajas como la posibilidad de trabajar en un amplio rango de frecuencias, configurar diferentes anchos de banda y reducir los costes de desarrollo (necesita menos hardware especializado), lo que ha favorecido su creciente utilización tanto en el ámbito de la investigación como en aplicaciones industriales y de comunicaciones.

En los últimos años, el uso de vehículos aéreos no tripulados (UAV) ha experimentado un crecimiento significativo en ámbitos como la vigilancia, la inspección de infraestructuras, la agricultura de precisión, las comunicaciones y la defensa. Este aumento ha generado un mayor interés por el análisis y estudio de las señales de radiofrecuencia utilizadas por estos

sistemas para establecer la comunicación entre el dron y su estación de control. En este contexto, las tecnologías SDR desempeñan un papel fundamental, ya que permiten capturar, analizar, procesar e incluso transmitir este tipo de señales de forma flexible, convirtiéndose en una herramienta ampliamente utilizada tanto en investigación como en el desarrollo y validación de sistemas de comunicaciones inalámbricas.

Por otra parte, las señales de radiofrecuencia pueden ser convertidas a banda base y ser representadas digitalmente mediante muestras complejas en formato IQ (In-Phase y Quadrature), permitiendo guardar, analizar y transmitir información desde diferentes dispositivos inalámbricos a través de ondas radioeléctricas . Procesar estas muestras de manera correcta es fundamental para una correcta transmisión del mensaje original como preservar los datos iniciales .

En este trabajo se utilizan archivos I/Q procedentes de un conjunto de datos de señales de radiofrecuencia capturadas en una investigación orientada a la detección e identificación de vehículos aéreos no tripulados (UAV). Estos archivos contienen las muestras digitales de las señales intercambiadas por los drones y constituyen el punto de partida de la pipeline desarrollada en este proyecto. Debido a que fueron capturados con un hardware y unos parámetros específicos, es necesario adaptarlos para poder transmitirlos utilizando una plataforma SDR diferente.

A partir de estos puntos, surge el interés por desarrollar una cadena de procesamiento de datos para adaptar archivos I/Q para adaptarlos a una transmisión mediante SDR

1.2 Estado del arte

1.2.1 Visión global

La representación de señales mediante los componentes I/Q es uno de los fundamentos del procesamiento digital de señales en el mundo moderno y es usado globalmente en sistemas de comunicaciones inalámbricas, radares, drones y diferentes aplicaciones de radiofrecuencia.

La aparición de SDR ha supuesto un cambio de cara a los sistemas de telecomunicaciones tradicionales, ayudando a implementar muchas de las aplicaciones de radio directamente sobre software. Entre los transceptores más utilizados se encuentra el AD9361, ampliamente usado tanto en investigación como en desarrollo, debido a su gran versatilidad y reducido tamaño.

Varios trabajos de investigación se han centrado en el análisis, captura y generación de señales mediante SDR. Sin embargo, la adaptación y validación de señales I/Q reales para su transmisión mediante plataformas SDR es un área el cual no ha sido ampliamente explorada y de alto interés debido a la necesidad de garantizar la adaptabilidad y fiabilidad de las muestras y la compatibilidad con el sistema a usar posteriormente.

Otro punto de interes en cuanto a SDR ya no es solo el sector de las telecomunicaciones, sino también el sector de defensa. En las guerras modernas estamos viendo un claro cambio de estrategias, anteriormente se elegía vehículos y métodos muy robustos con un precio muy alto, y ahora se está cambiando el enfoque hacia métodos de coste reducido más versátiles como pueden ser los drones. Esto ha creado un auge en la necesidad de investigación y desarrollo de drones.

Una de las piezas fundamentales de un dron inalámbrico es SDR, por lo cual el desarrollo de sistemas basados en SDR tienen un valor bastante importante a nivel nacional en diferentes sectores.

1.2.2 Visión personal

A nivel personal, he tenido contacto con SDR dos veces.

La primera fue trabajando en la actualización de radios destinadas al ejército las cuales utilizaban tecnología SDR.

Allí fui conociendo como funcionan por dentro, que archivos utilizaban, diferentes métodos de transmisión y que las piezas formaban parte de la misma, como puede ser la FPGA (Field Programmable Gate Array) la cual se encarga de generar las señales a partir de circuitos programables. Todo esto me ayudó a comprender como funcionaba SDR y me generó un interés por las mismas

Más tarde pase a trabajar en desarrollo de software utilizando tecnología SDR. En este trabajo fue donde pasé del punto teórico al práctico, familiarizadome con las diferentes tecnologías utilizadas y el funcionamiento de diferentes herramientas utilizadas en la pipeline (Como puede ser libiio y GNU Radio).

Estos dos contactos generaron un gran interés por SDR y me dieron una gran base teórica y practica para poder desarrollar una pipeline basada en SDR con herramientas que son relevantes hoy en dia el la industria, siguiendo buenas prácticas de desarrollo y utilizando una estructura profesional.

1.3 Problema a solucionar

Los archivos I/Q utilizados como entrada de la pipeline proceden de un conjunto de datos capturado con una plataforma SDR distinta de la utilizada en este proyecto. Esto hace que sus características no sean directamente compatibles con el sistema de transmisión utilizado, ya que no poseen las mismas especificaciones de formato de almacenamiento de las muestras, la frecuencia de muestreo y la representación numérica.

En este caso, los archivos originales almacenan las muestras complejas en formato Complex64, es decir, cada muestra I/Q está representada mediante dos valores de coma flotante de 32 bits (uno para la componente I y otro para la componente Q) mientras que la plataforma SDR utilizada requiere muestras enteras de 16 bits (Int16) donde las componentes I y Q se representane 16 bits intercalados en el orden I-Q. Además, la frecuencia de muestreo debe ajustarse a un valor compatible con las capacidades del dispositivo.

Por ello, resulta necesario desarrollar una etapa de procesamiento que adapte automáticamente estos archivos al formato requerido, asegurando al mismo tiempo que las características fundamentales de la señal original se conserven durante todo el proceso de conversión y transmisión.

Aunque este trabajo utiliza un conjunto de datos específicos y una plataforma SDR basada en el transceptor AD9361, la pipeline desarrollada no está limitada únicamente a este caso. Su diseño permite adaptar otros archivos I/Q siempre que se ajusten los parámetros de procesamiento, como la frecuencia de muestreo, la frecuencia central o el formato de salida requerido por el dispositivo de transmisión. De este modo, la solución desarrollada sirve como base reutilizable que puede adaptarse con facilidad a diferentes conjuntos de datos y plataformas SDR.

1.4 Objetivos

El objetivo principal de este Trabajo de Fin de Grado es diseñar y desarrollar una pipeline funcional de procesamiento y transmisión de señales I/Q, en la cual se adapte archivos digitales de entrada para ajustarse a nuestro método de transmisión , en este caso específico, para ser utilizada en una plataforma SDR.

Para lograr este objetivo principal, se subdivide en los siguientes objetivos específicos:

- Analizar la estructura y características de los archivos I/Q originales.
- Diseñar una etapa de procesamiento digital que permita adaptar los datos al formato requerido por el hardware de transmisión.

- Desarrollar e implementar mecanismos de filtrado, remuestreo y conversión de formato (asegurando así la integridad del mensaje original)

- Desarrollar una herramienta capaz de generar archivos compatibles con la plataforma SDR utilizada

- Implementar un sistema de transmisión mediante el uso de una SDR.

- Analizar las señales obtenidas mediante diferentes métodos de visualización y medida.

- Validar que la información transmitida mantiene las características fundamentales de las señales originales.

1.5 Alcance y limitaciones

Este trabajo se centra en un sistema completo de procesamiento de señales I/Q y su posterior transmisión mediante el uso de una plataforma SDR basada en el transceptor AD9361.

La solución tiene de alcance:

- El tratamiento de archivos I/Q digitales.
- El procesamiento de las señales mediante técnicas de DSP (Digital Signal Processing) .
- La representación visual de los archivos mediante Matlab.
- La conversión de formato a muestras enteras de 16 bits.
- La transmisión de los archivos ya procesados mediante una plataforma SDR.
- La validación de las señales mediante herramientas software y equipos de medida.

No obstante, existen ciertas limitaciones de este proyecto:

- El proyecto ha sido desarrollado usando un conjunto concreto de señales y parámetros de transmisión, por lo cual, si fuese a ser usado con otras señales o medios de transmisión (Una SDR diferente por ejemplo o unos datos de entrada alternativos) el código fuente debería ser adaptado para funcionar con los nuevos archivos de entrada o parámetros.

- La validación del sistema se realiza a nivel espectral y de comportamiento de señal, sin implementar un sistema completo de recepción y demodulación. Ya que para esto se necesitaría 2 plataformas SDR y solo dispongo de una.

- El estudio se centra en la transmisión mediante una ANTSDR con transceptor AD9361, por lo que requeriría adaptación para transmitir con otros dispositivos.

- No se tratan temas como los protocolos de comunicación utilizados sino únicamente el procesamiento digital y la transmisión a nivel físico.

2- Base teórica

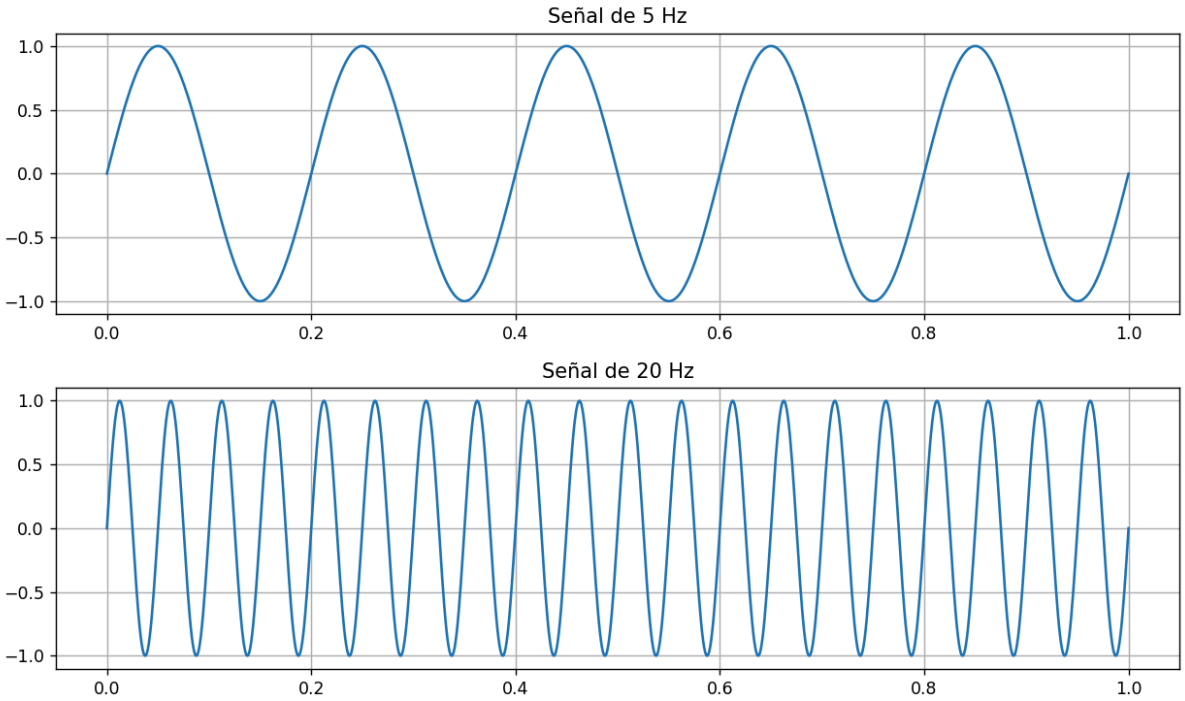
2.1 Radiofrecuencia (RF)

2.1.1 Conceptos

La radiofrecuencia es una rama de las telecomunicaciones que estudia la generación, transmisión y recepción de señales electromagnéticas dentro de un rango del espectro electromagnético específico. Estas señales nos permiten transmitir información a través del espacio sin necesidad de un medio físico portador. Esta es la base de numerosos sistemas de comunicación actuales (WiFi, teléfonos, satélites…).

Uno de los parámetros fundamentales de una señal de radiofrecuencia es la frecuencia portadora, medida en hercios (Hz), que indica el número de ciclos que una onda completa por segundo. La elección de la frecuencia portadora determina aspectos importantes del sistema de comunicaciones, como las características de propagación de la señal, las pérdidas durante la transmisión, el ancho de banda disponible y el tamaño de las antenas, ya que estas deben diseñarse en función de la longitud de onda correspondiente. Por ello, los distintos rangos de frecuencia se destinan a aplicaciones específicas según sus características.

Comparación de señales con diferentes frecuencias: Eje X: Tiempo (s) Eje Y: Amplitud (a)



Aquí podemos observar dos señales con diferentes frecuencias, la que tiene la frecuencia más alta oscila de un número mayor de veces por segundo.

Este comportamiento puede justificarse mediante la ecuación de propagación en espacio libre de Friis, que establece que las pérdidas de propagación aumentan con la frecuencia de la señal. Como la longitud de onda es inversamente proporcional a la frecuencia ,

$$\lambda = \frac{c}{f}$$

las señales de mayor frecuencia experimentan una mayor atenuación con la distancia, mientras que las de menor frecuencia pueden alcanzar mayores distancias de transmisión en las mismas condiciones. Por otro lado, las frecuencias más elevadas suelen disponer de un mayor ancho de banda disponible, lo que permite transmitir una mayor cantidad de información [2] [3].

La frecuencia portadora y el ancho de banda de una transmisión están directamente relacionados con el espectro radioeléctrico, que es el conjunto de frecuencias del espectro electromagnético destinadas a la transmisión de señales de radio, comprendidas normalmente entre 3 Hz y 300 GHz. Al tratarse de un recurso natural limitado, su uso se encuentra regulada a nivel nacional e internacional por diferentes organismos, como la Unión Internacional de Telecomunicaciones (UIT), que asignan las distintas bandas de frecuencia a servicios y aplicaciones específicas, como por ejemplo la SM.328 [4].

Es importante destacar que no todas las bandas de frecuencia pueden utilizarse libremente para transmitir. Muchas de ellas están reservadas para servicios específicos, como las comunicaciones móviles o sistemas aeronáuticos, por lo que su uso requiere de una autorización.

Sin embargo, existen determinadas bandas de uso libre, conocidas como bandas ISM (*Industrial, Scientific and Medical*), destinadas a aplicaciones industriales, científicas y médicas, que también son utilizadas por varias tecnologías inalámbricas. La elección de una frecuencia de transmisión debe tenerse en cuenta la normativa actual y las bandas autorizadas para evitar interferencias con otros servicios de radiocomunicaciones [5] .

Dentro de este espectro, otro parámetro fundamental es el ancho de banda, que representa el rango de frecuencias ocupado por una señal y que contiene la información útil transmitida. Por ejemplo, si una señal ocupa un rango entre 5790 MHz y 5810 MHz, su ancho de banda será:

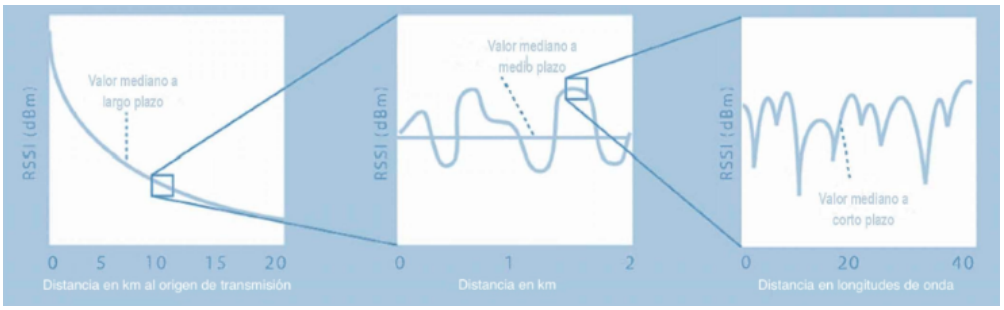
$$BW(BandWidth) = 5810 - 5790 = 20\ MHz$$

Cuanto más ocupe más información podremos transmitir pero también ocuparemos más rango espectral y necesitaremos una frecuencia de muestreo más alta para así poder mantener una resolución aceptable.

La frecuencia de muestreo se refiere a la cantidad de muestras por segundo que se toman al convertir una señal de audio analógico a digital. Cuantas más muestras tomemos mayor será la resolución de nuestro audio.

2.1.2 Sistemas de comunicación

Los sistemas de comunicación se componen de 3 elementos: Transmisor, canal y receptor. El transmisor se encarga de generar la señal y la adapta a las medidas necesarias para su transmisión. Después, la señal atraviesa el medio de transmisión, donde se ve afectada por el ruido, interferencias, pérdidas en la propagación y otros fenómenos físicos como la difracción y la dispersión de onda. Finalmente, el receptor capta la señal y la procesa para obtener la información que contenga.



Los sistemas modernos poseen un sistema de procesamiento de señales interno que les permite filtrar, modular y analizar las señales, aunque muchos de estos todavía están en desarrollo.

En este trabajo utilizaremos de transmisor una tarjeta SDR la cual transmitirá en las señales la información de nuestros archivos I/Q procesados para adecuarlos a nuestro dispositivo. El canal utilizado será un cable coaxial, y el receptor un medidor de espectro el cual nos ayudará a analizar la señal recibida.

2.2 Representación I/Q

La representación I/Q es fundamental dentro de los sistemas modernos de comunicación digital y de las plataformas de Software Definido por Radio (SDR).

Este método nos permite describir una señal de radiofrecuencia utilizando 2 componentes (I y Q), lo cual facilita el almacenamiento, transmisión y procesamiento digital de las mismas.

En vez de trabajar con una señal de alta frecuencia, los sistemas digitales representan la información con dos componentes de bajas frecuencias, In-Phase (I) y Quadrature (Q). Al combinar ambos componentes obtenemos la reconstrucción de la amplitud y la fase de la señal original. Así se logra almacenar señales complejas de manera digital.

2.2.1 Componentes I/Q

Los componentes I y Q como he mencionado representan la fase (I) y la amplitud (Q) y ambos son valores numéricos que nos permiten saber el estado instantáneo de la señal en cualquier instante. Debido a esto los sistemas de procesamiento digital de señales (DSP - Digital Signal Processing) trabajan con secuencias de números complejos en lugar de trabajar directamente con señales de radiofrecuencia.

En el formato I/Q tenemos dos señales simultáneas, pero la segunda (Q) tiene un desfase de 90° respecto a la primera, de esta manera estamos transmitiendo dos señales en un mismo archivo sin que interfieran entre ellas. Este desfase se representa mediante las siguientes ecuaciones:

$$I(t) = A \cos(\omega t)$$

$$Q(t) = A \operatorname{sen}(\omega t)$$

$$s(t) = I(t) \cos(\omega t) - Q(t) \operatorname{sen}(\omega t)$$

Aquí, A representa la amplitud y ω la frecuencia angular. Se podría decir que lo que están transmitiendo realmente es, I una coordenada horizontal y Q una coordenada vertical. La combinación de ambas determina el estado de la señal en ese instante

2.2.2 Amplitud y fase

Otra ventaja de el formato I/Q es que nos permite saber la amplitud y la fase de la señal a partir de sus componentes de la siguiente manera:

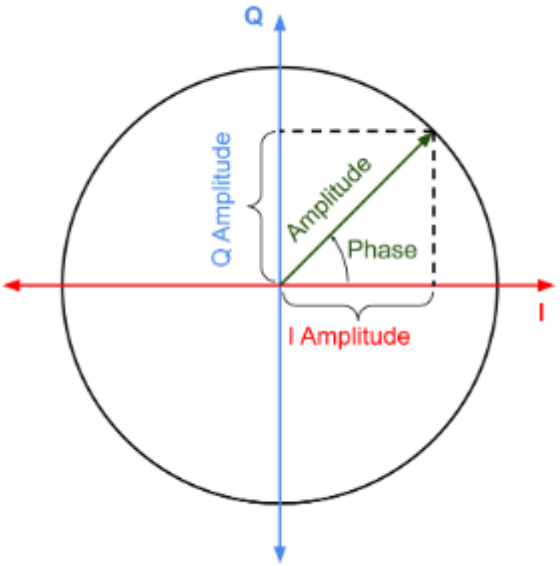
Amplitud: $$A = \sqrt{I^2 + Q^2}$$

Fase: $$\phi = \tan^{-1}\left(\frac{Q}{I}\right)$$

En la amplitud nos viene definida la intensidad de la señal y en la fase nos describe su posición actual dentro del ciclo. Por lo que a partir de los componentes IQ podemos reproducir la señal original entera, sin necesidad de conocer la señal portadora.

2.2.3 Plano I/Q

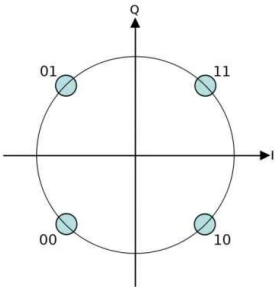
Las muestras I/Q se representan gráficamente mediante un plano cartesiano llamado plano I/Q. En éste, el valor I toma el eje horizontal y el valor Q toma el eje vertical, por lo que cada muestra compleja toma un punto dentro de nuestro plano:



Como podemos ver en la imagen, la distancia desde el punto hasta el origen representa la amplitud, Y el ángulo formado respecto al eje horizontal nos da la fase.

El plano I/Q es la base de los diagramas de constelación empleados en distintas modulaciones digitales como pueden ser QPSK y QAM.

Constelación QPSK:



2.3 Modulación

La modulación es el proceso que sirve para modificar una o varias propiedades de una onda portadora con el objetivo de transmitir información a través de un medio de comunicación. Dependiendo del sistema utilizado, pueden modificarse parámetros como la amplitud, la

frecuencia o la fase de la portadora, permitiendo adaptar la señal al canal de transmisión y mejorar la eficiencia de la comunicación [6].

Existen dos grandes categorías de modulación: la modulación analógica y la modulación digital. Mientras que la primera representa información continua, la modulación digital utiliza símbolos específicos para representar información binaria, siendo la base de los sistemas modernos de comunicaciones inalámbricas. Dentro de este grupo se encuentra la modulación QAM (Quadrature Amplitude Modulation), utilizada ampliamente en tecnologías como Wi-Fi, LTE o 5G y que se describe en el siguiente apartado.

2.3.1 Modulación digital

Los dispositivos digitales de comunicación trabajan con datos en formato binario. Para lograr esto se utilizan diferentes métodos de modulación digital, que tienen como objetivo representar cadenas de bits dentro de una señal portadora mediante variaciones en la misma [7].

A diferencia de la modulación analógica, en la cual se modifica la amplitud o la frecuencia de la señal, en la modulación digital se usan estados predefinidos para modificar la información, cada uno de estos estados puede representar uno o varios bits dependiendo del tipo de modulación elegida.

Esta es la base de numerosos sistemas modernos como la Wi-Fi o los satélites.

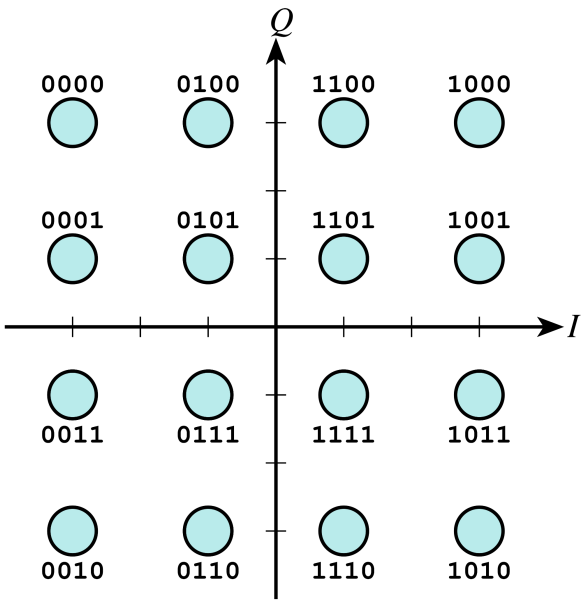
2.3.2 QAM (Quadrature Amplitude Modulation)

La modulación QAM combina la amplitud y la fase para optimizar la cantidad de información transmitida. Debido a esta optimización, es una de las técnicas de modulación más utilizadas en los sistemas de comunicación modernos.

QAM es el sistema de modulación más avanzado respecto a I/Q, ya que utiliza tanto la amplitud como la fase simultáneamente para generar símbolos a lo largo de un plano I/Q. Cada símbolo ocupa una posición dentro de una constelación lo cual representa un conjunto de bits determinados.

Hay diferentes tipos de modulación QAM dependiendo de la cantidad de bits usados en la constelación.

Por ejemplo en 16-QAM tenemos 4 bits por símbolo:



Podemos ver claramente que cada símbolo ocupa un valor dentro de nuestra constelación el cual puede ser traducido a un conjunto de 4 bits. Si estuviésemos usando 256-QAM cada símbolo corresponde a 8 bits.

A medida que aumentamos el número de símbolos disponibles también aumentamos la información transmitida, pero al estar más cerca unos de otros, aumenta la sensibilidad al ruido o interferencias.

Una vez ya sabemos como los archivos I/Q transmiten información y representan señales mediante una modulación, es posible aplicar diferentes técnicas de procesamiento digital de

señales (DSP) para filtrar, analizar, remuestrear o adaptar estas señales a las necesidades de nuestro sistema de transmisión.

2.4 Procesamiento Digital de Señales (DSP)

El procesamiento Digital de Señales (DSP) se trata de la manipulación y análisis de señales representadas digitalmente. De esta manera, se filtra, modifica y analiza la señal mediante diferentes algoritmos diseñados en software o hardware especializado.

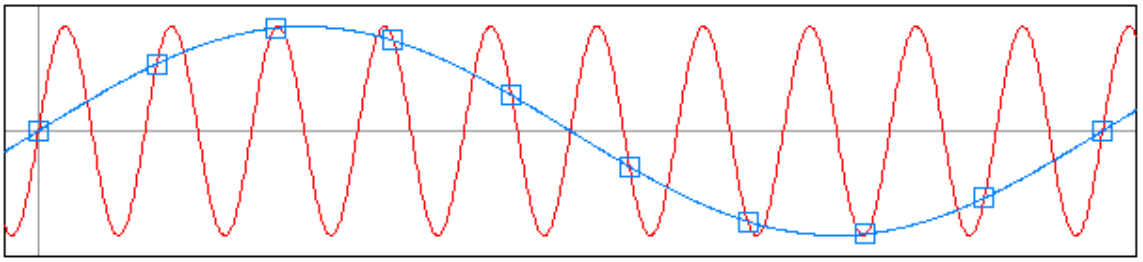
Esto es un proceso fundamental para adaptar la señal a nuestros requisitos para una transmisión óptima, funcional y con el menor número de interferencias. En este trabajo se usa DSP tanto para hacer que nuestras muestras sean compatibles con nuestra SDR como para asegurarnos de mantener la fidelidad de los archivos originales y perder la menor cantidad posible de información.

Las señales de radiofrecuencia son de naturaleza continua. En los sistemas SDR, para poder procesarlas mediante software, es necesario convertirlas previamente en una secuencia de muestras digitales mediante un proceso denominado muestreo. La cantidad de muestras obtenidas por segundo recibe el nombre de frecuencia de muestreo (fs).

Para que una señal pueda reconstruirse correctamente a partir de sus muestras, la frecuencia de muestreo debe cumplir el teorema de Nyquist-Shannon, que establece que esta debe ser, al menos, el doble del ancho de banda de la señal:

$$2B \leq fs$$

donde (fs) representa la frecuencia de muestreo y (B) el ancho de banda de la señal.



Por ejemplo, si una señal ocupa un ancho de banda de 20 MHz, la frecuencia de muestreo deberá ser, como mínimo, de 40 MSPS para evitar pérdidas de información durante el proceso de digitalización.

Cuando esta condición no se cumple, las componentes espectrales de la señal se solapan, produciendo un fenómeno llamado aliasing, que introduce una distorsión irreversible en la representación digital de la señal. En la imagen se puede observar cómo, al utilizar un número insuficiente de muestras, la reconstrucción de la señal no representa a la señal original.

Por este motivo, siempre que se reduzca la frecuencia de muestreo resulta necesario aplicar previamente un filtro anti-aliasing, cuya función es eliminar las componentes de frecuencia que no pueden ser representadas correctamente tras el remuestreo y evitar así la aparición de aliasing.

3- Tecnología y herramientas utilizadas

En el desarrollo de esta pipeline, he usado diversas tecnologías y herramientas enfocadas al procesamiento y transmisión de señales, tanto software como hardware. En este capítulo nombraré las tecnologías utilizadas y su funcionamiento y utilidad dentro del proyecto.

3.1 Software

3.1.1 Python

Python es un lenguaje de alto nivel utilizado en numerosos sectores como puede ser investigación o industrial. Es una gran elección debido a su fácil uso y numerosas bibliotecas, contando con varias enfocadas a procesamiento de señal.

Dentro del mundo de procesamiento digital se ha vuelto una herramienta muy usada debido a que al tener bibliotecas especializadas permite aplicar algoritmos complejos de manera sencilla y eficaz.

En este proyecto ha sido la herramienta principal a la hora de procesar señales I/Q . Mediante diferentes scripts se ha ido aplicando filtros digitales, remuestreo y conversión de formato, diferentes operaciones necesarias para asegurar la compatibilidad y fidelidad e nuestras musetras.

También se ha utilizado de cara a visualizar las muestras con representaciones en el dominio temrporal y en el dominio frecuencial, facilitando la validación de los resultados.

Bibliotecas usadas:

- NumPy: manejo de matrices y operaciones numéricas [8].
- SciPy: implementación de algoritmos de DSP [9].
- Matplotlib: representación gráfica de datos y espectro [10].

3.1.1 Matlab

Python no se ha usado solo sino que mediante Matlab hemos logrado unir ambos para un procesamiento de señales y representación visual más eficiente.

Matlab es una herramienta utilizada en ingeniería para análisis numérico, procesamiento de señales y muchos otros ámbitos (Matlab R2026a) [11].

Sirvió en el proyecto como herramienta complementaria para el análisis y validación de resultados obtenidos mediante Python, así como la comparación temporal y espectral de las señales procesadas.

Matlab y Python nos han proporcionado un entorno flexible para el procesamiento y verificación de los procesos realizados.

3.1.2 Entorno de transmisión

Linux es un sistema operativo muy modificable con alta flexibilidad. Tiene una alta compatibilidad con sistemas embebidos y SDR sirviendo como entorno de desarrollo especializado.

Gran parte de los sistemas SDR están enfocados a Linux por lo cual existen varias bibliotecas para el mismo que facilitan la comunicación entre el dispositivo hardware y nuestra plataforma.

Gracias al uso de Linux, he podido implementar librerías como GNU Radio [12] y Libiio [13] , las cuales son necesarias para el control y configuración del hardware utilizado, ambas son librerías especializadas en SDR.

La aplicación de transmisión ha sido desarrollada en lenguaje C debido a una mayor compatibilidad con las librerías mencionadas anteriormente, además de ser conocido por ser de bajo nivel teniendo mejor conexión con sistemas embebidos y SDR, al tener acceso directo a recursos hardware.

3.2 Hardware

3.2.1 SDR

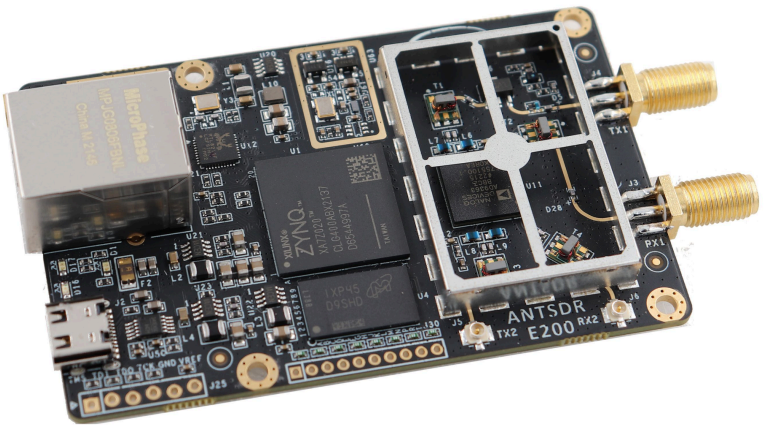
La Radio Definida por Software (SDR) es una tecnología que permite implementar mediante software diferentes funciones de sistemas tradicionales de radio que originalmente hacía un hardware dedicado.

En una radio tradicional, tienes numerosos elementos como moduladores, filtros o mezcladores que suelen estar implementados en circuitos específicos. En SDR, la mayoría de estas funciones pasan a ser implementadas digitalmente, lo que le da una mayor flexibilidad.

Básicamente SDR recibe muestra digitales y realiza los procesos necesarios para posteriormente convertirlas en señales de RF mediante conversores analogico-digital y diferentes circuitos modificables.

Esto nos permite recibir diferentes tipos de señales y ser capaz de procesarlas cambiando únicamente el software, manteniendo el hardware intacto

El modelo utilizado es una ANTSDR E200 con un transceptor AD9361 [14].



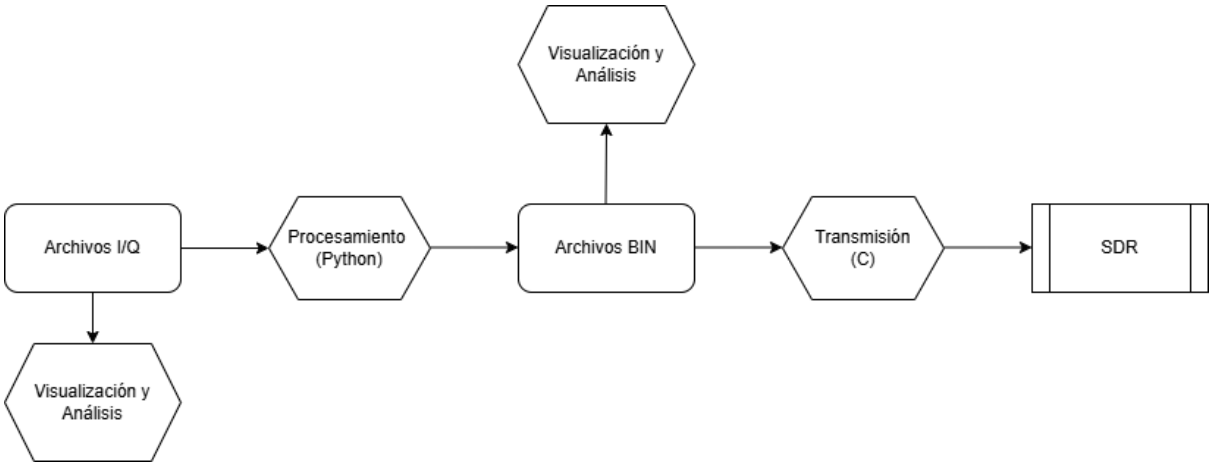
3.2.2 Analizador de espectro

Una vez terminada la pipeline y emitida la señal, conectamos un analizador de espectro para poder visualizar la señal transmitida y validar la pipeline.

De aquí se puede sacar información como banda base, ancho de banda y diferente información sobre el espectro. Gracias a este instrumento podemos hacer una comparación de nuestra señal previa al procesamiento y posterior, asegurando que no haya una pérdida de calidad significativa.

4- Diseño de la pipeline

Para lograr nuestro objetivo de procesamiento de datos y posterior transmisión necesitamos 2 módulos principales, uno de procesamiento de datos y señal, el cual utilizará Python debido a sus librerías especializadas en DSP. Y un módulo de transmisión el cual desarrollaremos en lenguaje C debido a su alta compatibilidad con SDR y hardware de bajo nivel. Además de eso, deberemos contar con un análisis y validación de datos en 3 puntos diferentes de la pipeline. Primero antes de procesar los datos, después una vez ya los procesamos, y por último una vez esté SDR transmitiendo los mismos, cabe destacar que el último paso debido a usar hardware físico está expuesto a ruido e interferencias.



Como se muestra en la imagen, la pipeline desarrollada se divide en las siguientes etapas:

- **Archivos I/Q:** La pipeline comienza con un conjunto de archivos I/Q que contienen las muestras complejas de señales de radiofrecuencia. Estos archivos son la entrada del sistema y representan la información que posteriormente será procesada y transmitida.

- **Visualización y análisis inicial:** Antes de realizar cualquier procesamiento, los archivos I/Q son analizados mediante Matplotlib para obtener representaciones espectrales de la señal. Este análisis permite comprobar el estado inicial de los datos y establecer una referencia para las futuras validaciones.

- **Procesamiento (Python):** Los archivos I/Q son procesados mediante una aplicación desarrollada en Python. En esta etapa se realizan las operaciones necesarias para adaptar los datos a los requisitos de la plataforma SDR, como el cambio de formato de Complex64 a Int16, el remuestreo de 100MSPS a 30.72, ya que ANTSDR solo soporta hasta 61MSPS, el filtrado anti-aliasing y la generación de los archivos BIN finales.

- **Archivos BIN:** Como resultado del procesamiento se obtienen archivos BIN compatibles con la plataforma SDR. Estos archivos contienen las muestras I/Q ya adaptadas y listas para ser transmitidas.

- **Visualización y análisis postprocesamiento:** Una vez generados los archivos BIN, se realiza un nuevo análisis espectral para comprobar que el procesamiento ha mantenido las características fundamentales de la señal original y que la conversión se ha realizado correctamente.

- **Transmisión (C):** Los archivos BIN se envían a una aplicación desarrollada en lenguaje C, que se encarga de configurar el transceptor AD9361 y transmitir las muestras I/Q. Para ello se usa la biblioteca libiio, que permite la comunicación con el hardware SDR.

- **SDR:** Finalmente, la plataforma SDR transmite la señal de radiofrecuencia a partir de nuestro archivo BIN. La salida RF analizarse mediante un analizador de espectro para verificar que la señal transmitida mantiene las características esperadas.

- **Validación final:** El último paso es comparar los resultados obtenidos en las tres etapas de análisis (antes del procesamiento, después del procesamiento y tras la transmisión). Si las representaciones espectrales mantienen la misma estructura y únicamente aparecen las variaciones esperadas debido al sistema de transmisión, se considera que la pipeline ha funcionado correctamente.

5- Implementación

5.1 Archivos I/Q

El primer paso de esta pipeline es seleccionar y hacer un análisis de los archivos a usar. En este caso se han seleccionado los archivos de una investigación de detección de drones [15] .

Dentro del conjunto de datos del estudio hay numerosas marcas de drones y varios datasets. Se ha seleccionado el dataset de DJI MAVIC 3 PRO [16] debido a que es un modelo ampliamente utilizado y también una marca muy presente dentro del mundo de los drones.

Una vez se ha importado el dataset dentro del proyecto, podemos escoger entre dos carpetas, una archivos con un nivel de ruido más alto y otra con archivos con un nivel de ruido muy

reducido, a esto se le llama Signal to Noise Ratio (SNR) y nos viene especificado dentro de los archivos.

Además de los archivos I/Q, dentro de las carpetas también encontramos un XML en el que nos dan información acerca de los archivos:

```xml
<?xml version="1.0" encoding="UTF-8"?>
<SignalHoundIQFile Version="1.0">
    <DeviceType>USRPX310</DeviceType>
    <Drone>DJI MAVIC 3 PRO</Drone>
    <SerialNumber>00010</SerialNumber>
    <DataType>Complex Float</DataType>
    <ReferenceSNRLevel>22</ReferenceSNRLevel>
    <CenterFrequency>5800000000.000</CenterFrequency>
    <SampleRate>100000000</SampleRate>
    <IFBandwidth>100000000</IFBandwidth>
    <ScaleFactor>90</ScaleFactor>
    <IQFileName>pack1.iq</IQFileName>
    <SampleCount>100000000</SampleCount>
</SignalHoundIQFile>
```

De aquí se saca bastante información útil. Lo primero es información sobre los drones de los cuales se han sacado estos archivos I/Q, el modelo de SDR que utilizan esos drones (USRPX310) y el número de serie, lo cual para este proyecto no es información tan relevante.

Pero seguido de esto, nos dicen el tipo de datos (ComplexFloat) lo cual es vital para la conversión a binario. También nos dan una referencia del SNR, seguido de la frecuencia central de la señal (5.8 GHZ). Dentro del mundo de los drones y otros dispositivos inalámbricos, las frecuencias más usadas suelen ser 5.8 GHz y 2.4GHz.

Después nos indican el Sample Rate, la frecuencia de muestreo (100MSPS). Y por último el número total de muestras por archivo, 100 000 000 muestras I/Q.

5.2 Visualización

Una vez se ha completado el análisis preliminar de los archivos I/Q pasamos a visualizarlos. Para esto desarrollamos un código en Python, este código implementará varias librerías especializadas en investigación y RF.

Utilizaremos NumPy para ayudarnos a realizar la lectura y manipulación de los archivos I/Q para asegurar su correcta visualización.

También dispondremos de Matplotlib la cual será la principal encargada de la visualización y la interfaz gráfica. Es una librería gráfica para poder representar y utilizar Matlab dentro de Python.

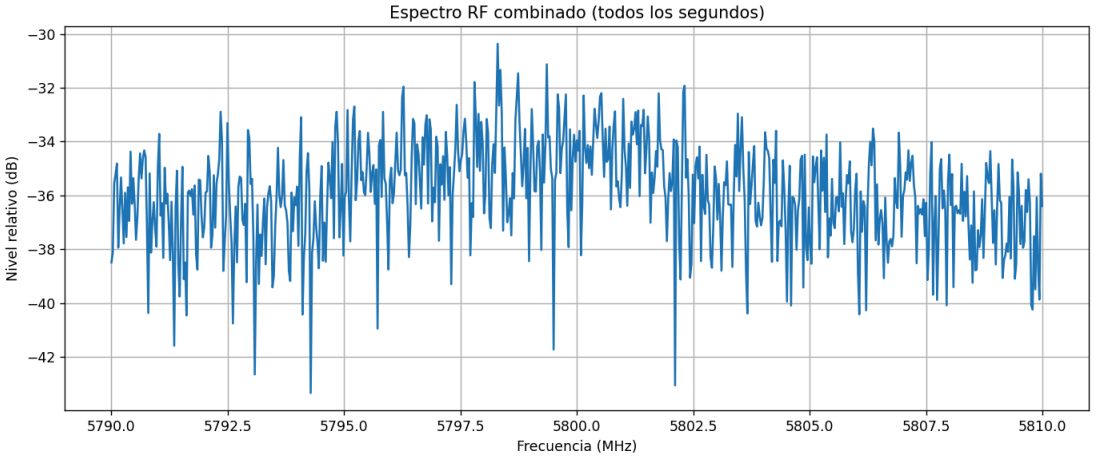
Y por último, SciPy para realizar diferentes algoritmos y cálculos complejos sobre las muestras.

El primer paso tras importar las librerías consiste en definir los parámetros necesarios para el análisis de la señal. Entre ellos se encuentran la frecuencia de muestreo, la frecuencia central, la resolución del ancho de banda (RBW), el rango de frecuencias que se desea visualizar (*SPAN*) y el número de bloques utilizados para calcular el espectro medio.

Una vez definidos estos parámetros, se selecciona el archivo I/Q que se desea analizar. El programa es capaz de leer tanto archivos originales en formato .iq como archivos procesados en formato .bin, pero para el formato .bin se deben hacer unas modificaciones que explicaré más tarde.

A continuación, la señal se divide en bloques de tamaño fijo y se calcula la Transformada Rápida de Fourier (FFT) [17] sobre cada uno de ellos. La FFT permite transformar la señal del dominio temporal al dominio frecuencial, mostrando cómo se distribuye la energía entre las distintas frecuencias que la componen. Antes de realizar este cálculo se aplica una ventana de Hann para suavizar los extremos de cada bloque y obtener una representación del espectro más estable y precisa.

El proceso se repite sobre un número determinado de bloques consecutivos y, posteriormente, se calcula el promedio de todos los espectros obtenidos. Finalmente, se representa el espectro de la señal en decibelios relativos (dB), obtenidos a partir del logaritmo del módulo de la transformada. Estos valores permiten comparar la distribución espectral y la potencia relativa de los distintos componentes de la señal, aunque no representan una medida absoluta de potencia como el dBm.



5.3 Procesamiento de archivos

El siguiente paso tras visualizar las muestras I/Q es procesarlas para asegurar su compatibilidad con nuestro dispositivo SDR, para esto realizaremos varias conversiones.



En nuestra foto se muestran todos los pasos por los cuales pasa nuestro archivo para estar listo.

Primero seleccionamos nuestro archivo I/Q, al cual le aplicamos un filtro de paso bajo mediante un filtro FIR con ventana de Hanning.

El objetivo de este filtrado es limitar el contenido espectral de la señal y eliminar aquellos componentes de la frecuencia que podrían producir aliasing durante el proceso de remuestreo. De esta forma se garantiza que únicamente se conserve la información útil antes de adaptar la señal a la nueva frecuencia de muestreo.

Básicamente, como posteriormente vamos a hacer un remuestreo muy grande, estamos preparando los datos para que no se vean afectados al realizarlo, asegurándonos así de que estamos representando correctamente los datos originales.

Para ello, se utiliza la función firwin [18] de la librería SciPy, encargada de diseñar los coeficientes de un filtro FIR a partir de los parámetros especificados . Posteriormente, dichos coeficientes se aplican a la señal para eliminar las componentes de frecuencia no deseadas antes del proceso de remuestreo.

Una vez preparado el archivo, pasamos a hacer el remuestreo. La razón principal de este remuestreo es que nuestros archivos I/Q vienen originalmente en 100MSPS (Mega Samples Per Second, que significa básicamente Millones de muestras por segundo), y nuestra transceptor AD9361 (el usado por ANTSDR) no trabaja con frecuencias de muestreo tan altas, por lo cual lo tenemos que bajar a una frecuencia dentro de su rango (Menor de 61 MSPS), por lo cual se ha elegido la frecuencia 30.72 MSPS ya que es un estándar dentro de las telecomunicaciones.

Para realizar este remuestreo utilizaremos el algoritmo resample_poly [19] el cual nos permite realizar la conversión manteniendo las características fundamentales de la señal y reduciendo las posibles distorsiones asociadas al cambio de frecuencia de muestreo.

Tras el remuestreo pasaremos a la fase de normalización , para esto usaremos el valor máximo presente en la señal. Así se asegura que se use todo el rango antes de realizar la la

conversión a formato entero, para lograr evitar pérdidas de resolución o saturación durante la transmisión

Una vez normalizadas las muestras, las componentes I y Q se convierten individualmente a enteros con signo de 16 bits, es decir, separamos I y Q para adaptar las muestras al formato SDR ya que las recibe por separado.

Después, se desarrolla un algoritmo para almacenar I y Q de manera intercalada para seguir el formato requerido por el transceptor AD9361. Por lo que nuestro archivo binario contiene las muestras de esta manera:

I0 Q0 I1 Q1 I2 Q2 I3 Q3 …

Así la SDR puede leer las muestras directamente durante la transmisión.

Por último, se almacenan las muestras intercaladas en un archivo binario (.bin), obteniendo un archivo totalmente compatible con nuestro sistema SDR y también con el siguiente módulo encargado de enviar nuestro archivo a SDR para su transmisión.

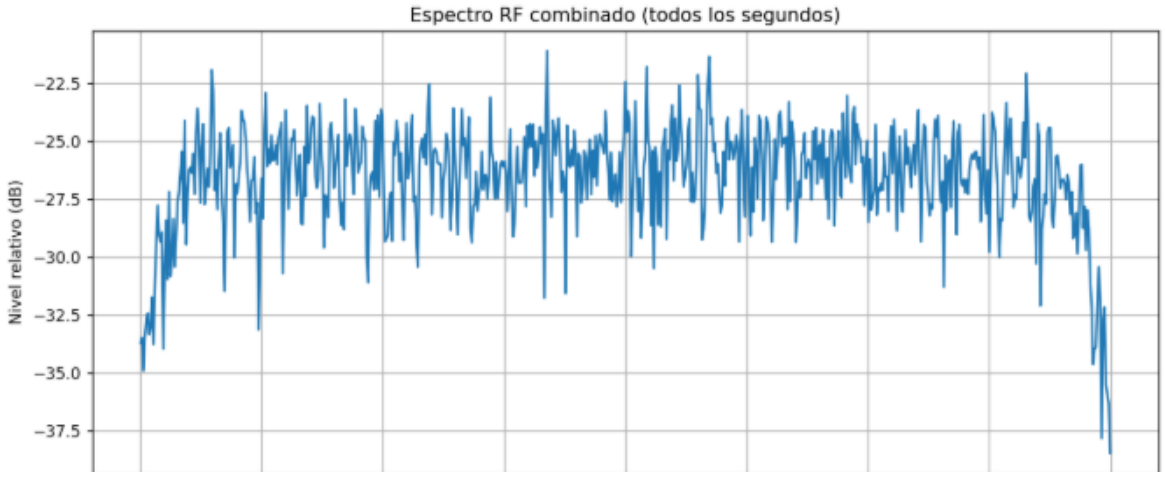
5.3 Validación (visualización)

El siguiente paso de la pipeline es validar que el archivo se haya procesado correctamente sin una pérdida significativa de datos. Para ello se utiliza el código que se ha desarrollado anteriormente, pero se deben de realizar algunas modificaciones para así poder visualizar archivos .bin en vez de .iq.

Las diferencias se encuentran en los parámetros y en la lectura de datos. Mientras que I/Q almacena las muestras complejas en formato Complex64, los archivos BIN contienen las componentes I y Q intercaladas en formato Int16. Por lo que antes de realizar el análisis es necesario reconstruir las muestras complejas, separando ambos componentes y normalizando sus valores para recuperar el rango de amplitud original.

Además, para la representación de los archivos BIN necesitamos cambiar los parámetros de frecuencia de muestreo, ya que al procesarlos han pasado de 100MSPS a 30.72MSPS.

Todos el resto del algoritmo utilizado para la representación visual queda igual.



6- Implementación de la transmisión

6.1 Configuración del entorno

Para realizar la transmisión, se ha decidido usar Linux por su compatibilidad con SDR. La manera que se ha configurado este entorno ha sido descargando un subsistema de Linux (Ubuntu) dentro del dispositivo principal (en el cual se ha desarrollado el código de Python). Por lo cual tenemos un sistema Windows con la parte de procesamiento de datos y un subsistema Linux con la parte de transmisión.

Para preparar correctamente el subsistema Linux, es necesario configurarlo para que pueda comunicarse con la SDR a través de la misma red local. La ANTSDR utilizada en este proyecto tiene asignada la dirección IP 192.168.1.10, por lo que el ordenador anfitrión debe estar configurado dentro de la misma subred (por ejemplo, 192.168.1.X con máscara 255.255.255.0). Al utilizar un subsistema Linux sobre Windows, no es necesario configurar

una interfaz de red independiente, ya que el subsistema utiliza la conectividad de red del sistema anfitrión para acceder al dispositivo SDR.

Una vez configurada la red, podemos hacer un ping a SDR para asegurarnos de que comuniquen correctamente.

```
pepinonico@PepinoNico:~$ sudo su
[sudo] password for pepinonico:
root@PepinoNico:/home/pepinonico# ping 192.168.1.10
PING 192.168.1.10 (192.168.1.10) 56(84) bytes of data.
From 192.168.1.2 icmp_seq=3 Destination Host Unreachable
64 bytes from 192.168.1.10: icmp_seq=5 ttl=63 time=2.49 ms
64 bytes from 192.168.1.10: icmp_seq=6 ttl=63 time=1.40 ms
64 bytes from 192.168.1.10: icmp_seq=7 ttl=63 time=2.12 ms
64 bytes from 192.168.1.10: icmp_seq=8 ttl=63 time=1.63 ms
64 bytes from 192.168.1.10: icmp_seq=9 ttl=63 time=1.76 ms
64 bytes from 192.168.1.10: icmp_seq=10 ttl=63 time=1.90 ms
64 bytes from 192.168.1.10: icmp_seq=11 ttl=63 time=2.35 ms
64 bytes from 192.168.1.10: icmp_seq=12 ttl=63 time=1.96 ms
64 bytes from 192.168.1.10: icmp_seq=13 ttl=63 time=1.61 ms
64 bytes from 192.168.1.10: icmp_seq=14 ttl=63 time=2.16 ms
64 bytes from 192.168.1.10: icmp_seq=15 ttl=63 time=1.85 ms
64 bytes from 192.168.1.10: icmp_seq=16 ttl=63 time=2.20 ms
```

Tras confirmar que las redes están configuradas correctamente, pasamos a importar los archivos a nuestro subsistema, para esto se seleccionan los archivos procesados por nuestro codigo Python, los copiamos, y buscamos donde se encuentra la carpeta del subsistema Linux, cuando la encontremos, creamos una carpeta y pegamos los archivos procesados.

Al configurar la red e importar los archivos, nuestro entorno ya estaría listo.

6.2 Desarrollo de código de transmisión

Lo primero a definir a la hora de explicar el desarrollo de este código es el pilar fundamental de el mismo Libiio, Libiio es una biblioteca que se encarga de toda la comunicacion entre el

ordenador y ANTSDR y esta biblioteca es compatible con el framework industrial I/O (IIO) de Linux.
Esta biblioteca proporciona una interfaz de alto nivel para acceder a los parámetros de configuración del transceptor AD9361 como a los buffers utilizados para el envío y recepción de muestras IQ.

Algunas de sus funciones más importantes son:

- Creación del contexto de comunicación con la SDR.
- Localización de dispositivos y canales disponibles.
- Configuración de parámetros de radiofrecuencia.
- Gestión de buffers de transmisión.
- Envío de muestras IQ al hardware.

Gracias a esta biblioteca no tenemos que acceder directamente al controlador del dispositivo, simplificando el desarrollo del software de transmisión.

Ya explicado Libiio, analizemos el funcionamiento del código de transmisión.

Lo primero que tiene este código es una definición de parámetros, los cuales son:

- Frecuencia central
- Frecuencia de muestreo
- Ancho de banda

Estos son modificables, en nuestro caso los valores que hemos establecido son, la frecuencia de muestreo que ya la definimos anteriormente, bajando de 100 a 30.72 ya que ANTSDR solo soporta hasta 61MSPS (30.72 MSPS) y de ancho de banda se escoge 10 MHz ya que al hacer un remuestreo reduciendo el numero de muestras necesitamos que nuestra frecuencia de muestreo (30.72 MSPS) sea mayor que el doble del ancho de banda, cumpliendo así con el Teorema de muestreo de Nyquist-Shannon que mencionamos anteriormente.

Una vez definido los parámetros, seleccionamos el archivo a transmitir y definimos los parámetros de red y hardware .Para ello se crea un contexto IIO indicando la dirección IP del dispositivo. Una vez establecida la comunicación, el programa localiza automáticamente los dispositivos internos necesarios para la transmisión, principalmente el transceptor AD9361.

Una vez detectado el hardware, el programa obtiene los canales correspondientes a la transmisión.

Por un lado se accede a los canales utilizados para configurar el transceptor (frecuencia local, ganancia y ancho de banda) y, seguido , se obtienen los canales digitales correspondientes a las componentes I (In-Phase) y Q (Quadrature).

La separación entre canales de configuración y canales de datos permite modificar los parámetros RF independientemente del flujo de muestras IQ que posteriormente será transmitido.

Después, el programa configura los principales parámetros del transmisor.

Asignamos los parámetros establecidos a nuestra SDR:

- Frecuencia portadora.
- Frecuencia de muestreo.
- Ancho de banda del filtro RF.

También se desactiva el generador DDS (Síntesis Digital Directa, el generador de señal interno de la SDR) del dispositivo para garantizar que la salida RF proceda exclusivamente del archivo IQ transmitido. Ya que si no se desactiva puede generar ruido no deseado modificando ligeramente el archivo que queremos transmitir

El siguiente paso es habilitar los canales que se van a usar para la transmisión

En esta fase se activan los canales correspondientes a las componentes I y Q (I -> voltage0 | Q -> voltage1) del transmisor. Así el dispositivo queda preparado para aceptar muestras del programa.

Una vez configurado el hardware, se crea un buffer de transmisión mediante libiio.

Este buffer sirve de memoria intermedia entre la aplicación y la SDR, permitiendo almacenar temporalmente un bloque de muestras antes de enviarlas al dispositivo.

Simultáneamente, el programa reserva memoria para almacenar los datos leídos desde el archivo BIN. Cada bloque contiene muestras IQ codificadas en formato int16 e intercaladas en el orden I-Q, para asegurar la compatibilidad con el transmisor AD9361.

El uso de buffers evita realizar una transferencia al hardware por cada muestra individual, mejorando notablemente la eficiencia del proceso de transmisión.

La transmisión se realiza mediante un bucle que permanece activo hasta que el usuario interrumpe la ejecución.

En cada iteración del bucle se realizan las siguientes operaciones:

1. Se lee un bloque de muestras IQ desde el archivo BIN.
2. Si se alcanza el final del archivo, la lectura vuelve automáticamente al inicio, permitiendo una transmisión continua.
3. Las muestras I y Q se copian al buffer gestionado por libiio.
4. El contenido del buffer se envía al dispositivo mediante una operación de escritura.

Este proceso se repite continuamente generando un flujo constante de muestras que son convertidas a RF mediante la SDR.

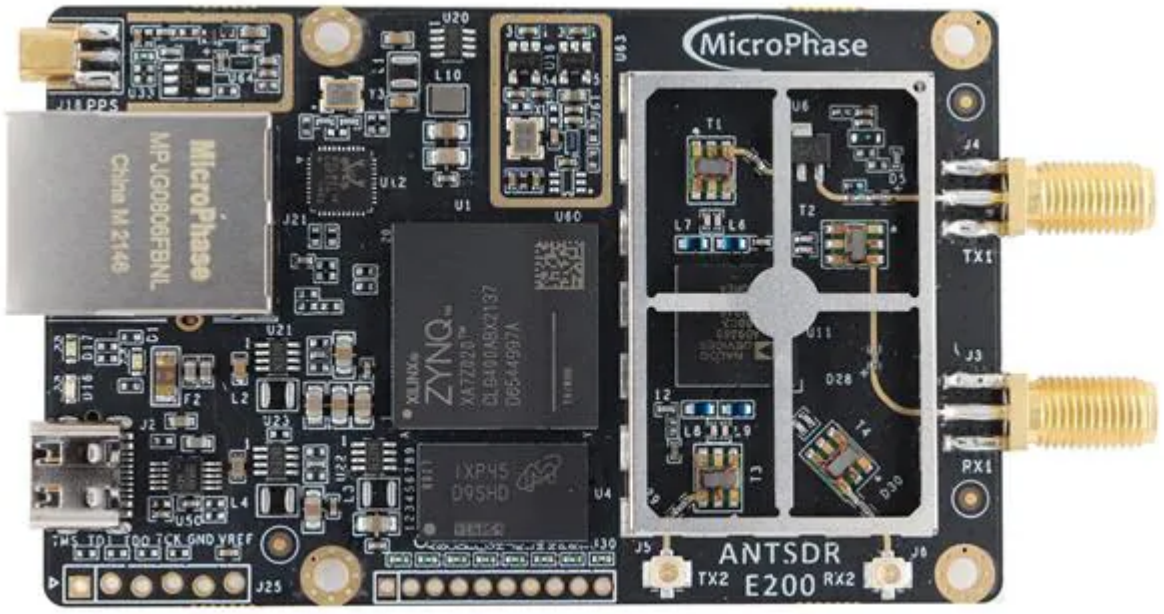
Cuando el usuario finaliza la ejecución, el programa sale del bucle principal y libera todos los recursos, se destruyen los buffers creados, se deshabilitan los canales de transmisión, se destruye el contexto IIO, se cierra el archivo de entrada y se libera la memoria reservada.

Gracias a esto nos aseguramos de que no queden recursos ocupados en el sistema y así finalizamos la aplicación de forma segura y controlada.

6.3 Transmisión RF

Para la transmisión utilizando el hardware hay que realizar unos sencillos pasos para conectar la SDR y el computador.

Debemos conectar ambas mediante un cable ethernet y usar un cable tipo C para la alimentación de la SDR.



A la izquierda de la tarjeta se puede observar la entrada del cable ethernet y del tipo C.

Una vez conectado esto, hacemos ping desde nuestro subsistema para asegurarnos de que se comunican.

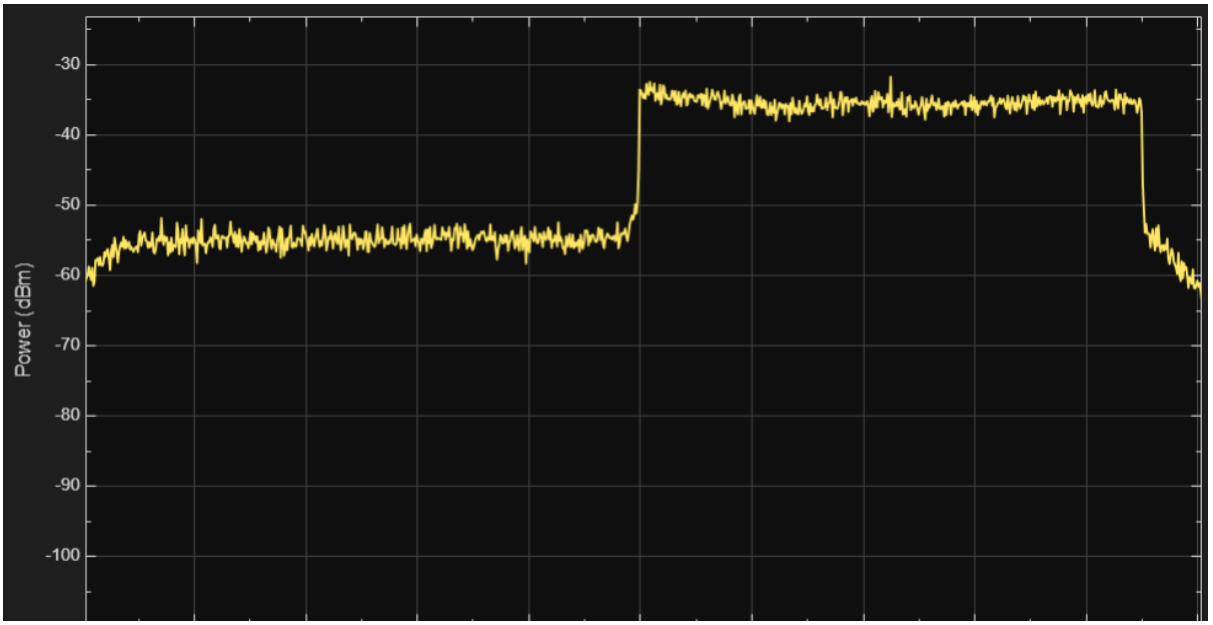
```
root@PepinoNico:/home/pepinonico# ping 192.168.1.10
PING 192.168.1.10 (192.168.1.10) 56(84) bytes of data.
From 192.168.1.2 icmp_seq=3 Destination Host Unreachable
64 bytes from 192.168.1.10: icmp_seq=5 ttl=63 time=2.49 ms
64 bytes from 192.168.1.10: icmp_seq=6 ttl=63 time=1.40 ms
```

Tras asegurarnos de que comunica, conectamos en la parte de la derecha, donde pone TX1 (transmisión), debemos conectar un cable coaxial el cual irá unido a un medidor de espectro o a otra SDR en el canal de recepción (RX1).

Una vez tenemos esto conectado tan solo tenemos que ejecutar nuestro código C especificando que archivo usar.

```
root@PepinoNico:/home/pepinonico/AD9361# ./ANTS 123T.bin
[OK] Archivo BIN abierto: 123T.bin
[OK] Contexto IIO: 192.168.1.10 Linux (none) 6.1.0-23073-gf3da30df6004-dirty
[OK] Dispositivos PHY y TX encontrados
[CFG] TX LO = 2000.000 MHz
[CFG] Sample rate = 30.72 MSPS
[CFG] RF BW = 20.00 MHz
[CFG] TX Gain = -5 dB
[OK] Canales TX I/Q habilitados
[OK] Buffer IIO creado (16384 muestras)
[TX] Transmitiendo (Ctrl+C para salir)...
```

Y finalmente analizamos los resultados obtenidos en nuestro medidor de espectro



6.2 Validación

La validación de la pipeline se ha realizado en tres etapas consecutivas:

1- Análisis del archivo IQ original, obteniendo la representación espectral de la señal antes del procesamiento.

2- Análisis del archivo procesado, tras la conversión de formato, el remuestreo y el resto de etapas de procesamiento digital.

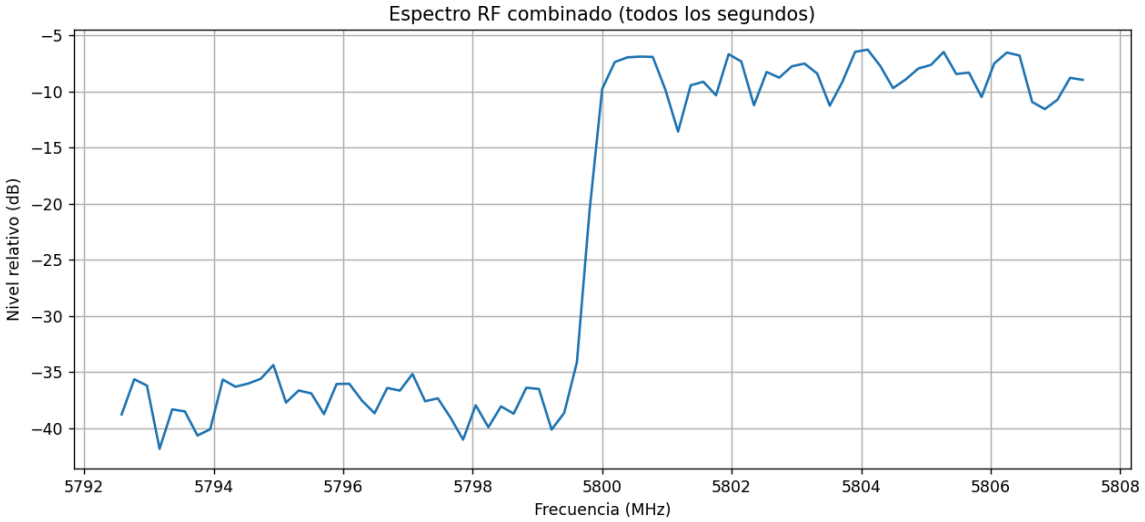
3- Análisis de la señal transmitida en RF, utilizando un analizador de espectro para observar la señal radiada por la SDR u otra SDR que reciba la señal.

El objetivo de esta validación es comprobar que las distintas etapas de la pipeline no alteran de forma significativa las características espectrales de la señal.

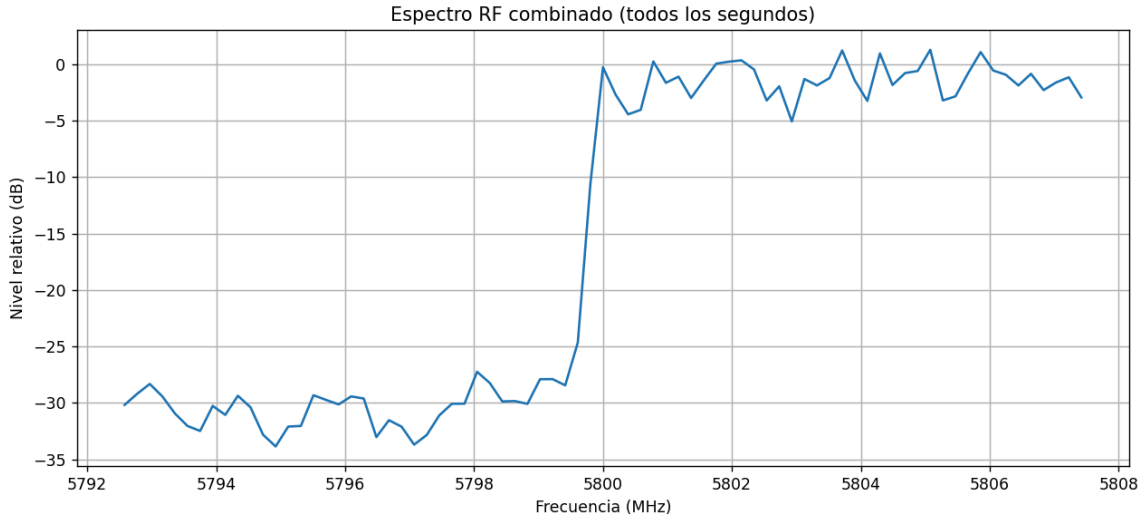
Como criterio de validación, se comparan las tres representaciones obtenidas, verificando que mantienen una distribución espectral similar, con la misma ocupación en frecuencia y una forma equivalente.

Aunque pueden existir pequeñas diferencias en amplitud debidas al proceso de transmisión(ruido e interferencias externas), las pérdidas del sistema o la calibración del instrumento de medida, la conservación de la forma general del espectro indica que el procesamiento y la transmisión se han realizado correctamente.

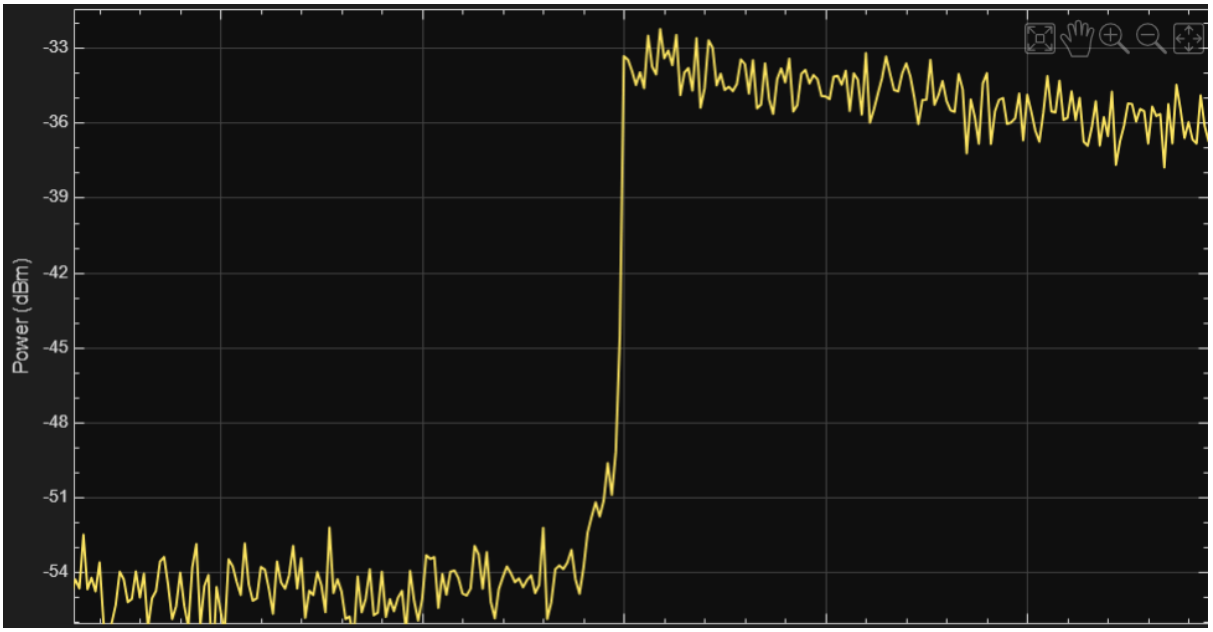
Espectro del archivo pack1_2-3s.iq (previo al procesamiento):



Espectro del archivo pack1_2-3s.bin (post-procesamiento):



Espectro de la señal transmitida mediante SDR (archivo usado: pack1_2-3s.bin):



Tras comparar las tres representaciones puede observarse que la forma del espectro se mantiene en todas las etapas del proceso. Las pequeñas variaciones de nivel observadas son esperables debido al sistema de transmisión y a las condiciones de medida, pero no afectan a la distribución espectral de la señal. Por tanto, puede considerarse que la conversión de los archivos IQ y su posterior transmisión mediante SDR mantiene las características de la señal original.

Etapa	Forma de espectro	Ancho de banda	Resultado
IQ	Original	10MHz	Correcto
Bin	Similar	10MHz	Correcto
RF	Similar	10 MHz	Correcto

7- Mejoras futuras

7.1 Soporte para múltiples SDR

Actualmente la pipeline ha sido desarrollada y validada para una ANTSDR basada en el transceptor AD9361. Como trabajo futuro sería interesante adaptar la aplicación para soportar diferentes plataformas SDR, como USRP, PlutoSDR, LimeSDR o HackRF, permitiendo seleccionar automáticamente la configuración adecuada para cada dispositivo.

7.2 Automatización de parámetros

Actualmente algunos parámetros se tienen que introducir manualmente (frecuencia de transmisión, frecuencia de muestreo, ancho de banda, etc.).

Como mejora futura podría desarrollarse un sistema que leyese automáticamente estos parámetros desde el XML asociado a los archivos IQ o desde una configuración externa.

De esta forma la pipeline sería completamente automática.

7.3 Interfaz gráfica

El proyecto está implementado como una pipeline compuesta por diferentes herramientas y programas independientes, donde cada etapa del proceso (selección de archivos, procesamiento, configuración de parámetros y transmisión mediante SDR) se ejecuta de forma separada. Esto hace que el usuario interactúe con distintos entornos y aplicaciones.

Como línea de trabajo futura, sería interesante desarrollar una aplicación con interfaz gráfica que integrase todas las funcionalidades de la pipeline en un único entorno. De esta forma, el usuario podría realizar todo el flujo de trabajo desde una única aplicación, incluyendo la selección de archivos IQ, la configuración de los parámetros de procesamiento, la conversión a formato compatible con la SDR, la configuración de los parámetros de transmisión y el control del dispositivo SDR para la emisión de la señal.

Esta integración mejoraría notablemente la usabilidad del sistema, reduciría la complejidad operativa y facilitaría su utilización por usuarios con menor experiencia técnica, además de proporcionar una plataforma más completa, integrada y centralizada.

8- Conclusión

El objetivo principal de este Trabajo Fin de Grado ha sido diseñar e implementar una pipeline capaz de transformar archivos IQ en un formato compatible con una plataforma SDR, permitiendo su posterior transmisión mediante radiofrecuencia.

Objetivos alcanzados:

- Desarrollo de la pipeline.
- Conversión IQ → BIN.
- Visualización IQ/BIN
- Adaptar archivos a formato SDR.
- Transmisión SDR mediante código en C (libiio)
- Configuración de la ANTSDR.
- Validación

Contribuciones del proyecto

En este proyecto se ha logrado desarrollar una pipeline funcional capaz de transformar archivos IQ en un formato compatible con una plataforma SDR y transmitir posteriormente la señal mediante radiofrecuencia. Para ello se han integrado de manera exitosa diferentes tecnologías y herramientas, como Python y Matlab para el procesamiento digital de señal y su visualización y análisis, C para el control del hardware y la biblioteca libiio para la comunicación con la SDR. Y se ha logrado medir la señal RF en un espectro radioeléctrico con los parámetros esperados.

Limitaciones

- Solo soporta AD9361.
- Algunos parámetros siguen siendo manuales.
- La validación se ha realizado principalmente mediante comparación espectral.
- El sistema está pensado para señales offline.

Reflexión final

Este Trabajo Fin de Grado ha sido una oportunidad para aplicar de forma práctica conocimientos adquiridos durante el grado y ampliar la formación en áreas como el procesamiento digital de señal, las comunicaciones por radiofrecuencia y la programación de sistemas embebidos. El desarrollo del proyecto me ha permitido comprender el funcionamiento completo de una cadena de transmisión SDR, desde el tratamiento de los datos IQ hasta su emisión en radiofrecuencia.

El proyecto me ha mostrado la importancia de combinar conocimientos de distintas disciplinas para desarrollar una solución completa, integrando software de procesamiento, programación de bajo nivel y hardware especializado. La experiencia adquirida durante este trabajo me proporciona una base sólida para continuar profundizando en el ámbito de las comunicaciones digitales, las tecnologías SDR y el procesamiento digital de señales, áreas de gran interés y con una amplia proyección tanto en investigación como en el sector industrial.

9- Bibliografía

Referencias

[1] Wireless Innovation Forum, *Software Defined Radio*. https://www.wirelessinnovation.org/.

[2] International Telecommunication Union, ITU-R P.525-2: Calculation of Free-Space Attenuation, 1994.

[3] International Telecommunication Union, Link Budget, ITU-R Small Satellites Workshop, University of Vigo.

[4] Unión Internacional de Telecomunicaciones (UIT), *UIT-R SM.328: Espectros y anchuras de banda de las emisiones*.

[5] Ministerio para la Transformación Digital y de la Función Pública, *Cuadro Nacional de Atribución de Frecuencias (CNAF)*.

[6] S. Haykin, *Communication Systems*, 4th ed. Hoboken, NJ, USA: John Wiley & Sons, 2001, ch. 3.

[7] J. G. Proakis and M. Salehi, *Digital Communications*, 5th ed. New York, NY, USA: McGraw-Hill, 2008, ch. 5.

[8] NumPy Developers, *NumPy Documentation*.

[9] SciPy Developers, *SciPy Documentation*.

[10] Matplotlib Development Team, *Matplotlib Documentation*.

[11] MathWorks, *Signal Processing Toolbox Documentation*.

[12] GNU Radio Project, *GNU Radio Documentation.*

[13] Analog Devices, *libiio GitHub Repository.*

[14] Analog Devices, *AD9361 Reference Manual (UG-570)*.

[15] K. Frank *et al.*, *RFUAV: A Large-Scale RF Dataset for UAV Detection and Classification*, arXiv preprint arXiv:2503.09033, 2025.

[16] K. Frank, *RFUAV Dataset – DJI MAVIC 3 PRO*.

[17] Analog Devices, *The Scientist and Engineer's Guide to Digital Signal Processing, Chapter 12: Sampling Theorem.*

[18] SciPy Developers, *SciPy Documentation – scipy.signal.firwin*.

[19] SciPy Developers, *SciPy Documentation – scipy.signal.resample_poly*.

- Richard G. Lyons. *Understanding Digital Signal Processing*. 3rd Edition. Pearson Education, 2011.

- John G. Proakis, Dimitris G. Manolakis. *Digital Signal Processing: Principles, Algorithms and Applications*. Pearson.
- Simon Haykin. *Communication Systems*. Wiley.

10- Anexo

1 - Parámetros utilizados

SDR - ANTSDR AD9361 E200

Frecuencia TX - 2 GHz

Sample Rate - 30.72 MSPS

Ancho de banda - 20MHz

Formato de entrada - Complex64

Formato de salida - Int16

Sistemas operativos - Windows 11 y Ubuntu Linux

Librerias - Libiio , Matplotlib, scipy,numpy

2 - Manual de uso

1- Selección de archivos .iq a usar

2- Importar archivos dentro de la aplicación Python

3- Ejecutar código de visualización IQ

4- Ejecutar código de procesamiento de datos

5- Ejecutar código de visualización BIN

6- Importar archivos BIN en subsistema linux

7- Conexión hardware de SDR

7.1 - Cable Tipo C a alimentación

7.2 - Cable ethernet a PC

7.3 - Cable coaxial a medidor de espectro

8- Conectar red del sistema a la red de SDR

9- Test de comunicación(Ping)

10- Ejecutar código de transmisión con el archivo seleccionado

11- Visualizar señal mediante medidor de espectro